

EMSEdit: Efficient Multi-Step Meta-Learning-based Model Editing

Xiaopeng Li
National University of
Defense Technology
Changsha, China
xiaopengli@nudt.edu.cn

Shasha Li*
National University of
Defense Technology
Changsha, China
shashali@nudt.edu.cn

Xi Wang
National University of
Defense Technology
Changsha, China
wx2023@nudt.edu.cn

Shezheng Song
National University of
Defense Technology
Changsha, China
ssz614@nudt.edu.cn

Bin Ji
National University of
Defense Technology
Changsha, China
jibin@nudt.edu.cn

Shangwen Wang
National University of
Defense Technology
Changsha, China
wangshangwen13@nudt.edu.cn

Jun Ma*
National University of
Defense Technology
Changsha, China
majun@nudt.edu.cn

Xiaodong Liu
National University of
Defense Technology
Changsha, China
liuxiaodong@nudt.edu.cn

Mina Liu
KylinSoft
Changsha, China
a506lm@126.com

Jie Yu*
National University of
Defense Technology
Changsha, China
yj@nudt.edu.cn

Abstract

Large Language Models (LLMs) power numerous AI applications, yet updating their knowledge remains costly. Model editing provides a lightweight alternative through targeted parameter modifications, with meta-learning-based model editing (MLME) demonstrating strong effectiveness and efficiency. However, we find that MLME struggles in low-data regimes and incurs high training costs due to the use of KL divergence. To address these issues, we propose **Efficient Multi-Step Edit (EMSEdit)**, which leverages multi-step backpropagation (MSBP) to effectively capture gradient-activation mapping patterns within editing samples, performs multi-step edits per sample to enhance editing performance under limited data, and introduces norm-based regularization to preserve unedited knowledge while improving training efficiency. Experiments on two datasets and three LLMs show that EMSEdit consistently outperforms state-of-the-art methods in both sequential and batch editing. Moreover, MSBP can be seamlessly integrated into existing approaches to yield additional performance gains. Further experiments on a multi-hop reasoning editing task demonstrate EMSEdit's robustness in handling complex edits, while ablation studies validate the contribution of each design component. Our code is available at <https://github.com/xpq-tech/emsedit>.

CCS Concepts

• Computing methodologies → Natural language generation.

*Shasha Li, Jun Ma, and Jie Yu are the corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792357>

Keywords

Large Language Models, Meta-learning, Model Editing

ACM Reference Format:

Xiaopeng Li, Shasha Li*, Xi Wang, Shezheng Song, Bin Ji, Shangwen Wang, Jun Ma, Xiaodong Liu, Mina Liu, and Jie Yu*. 2026. EMSEdit: Efficient Multi-Step Meta-Learning-based Model Editing. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792357>

1 Introduction

Large Language Models (LLMs) have emerged as foundational infrastructure for a wide range of AI applications [17, 37]. Through extensive pretraining on diverse corpora followed by alignment, LLMs acquire rich world knowledge. However, the knowledge encoded in LLMs becomes fixed after training, making them unable to adapt to updates in the real world [34]. Re-training LLMs to reflect new knowledge is prohibitively expensive [4]. This issue is particularly critical in the context of the Web, which serves as the primary medium for real-time knowledge dissemination. While the Web evolves on a daily basis, LLMs remain static, leading to a growing gap between model knowledge and current information.

To address this challenge, recent studies have introduced **model editing** as an efficient paradigm for updating knowledge in LLMs [8, 16, 19, 21, 31, 39]. These approaches aim to make LLMs memorize new information by modifying only a small subset of parameters. Unlike vanilla fine-tuning, model editing typically computes a weight update that indirectly injects new knowledge into the model. For example, **locate-then-edit** methods perform least-squares updates on the feed-forward networks that are believed to store factual knowledge [9, 10, 15, 20], while **meta-learning-based model editing (MLME)** employs a lightweight hypernetwork to transform fine-tuning gradients and internal representations into effective weight updates [16, 21, 25]. Although locate-then-edit approaches achieve impressive editing performance, they suffer from low efficiency. For instance, in sequential editing tasks, AlphaEdit [8] (a

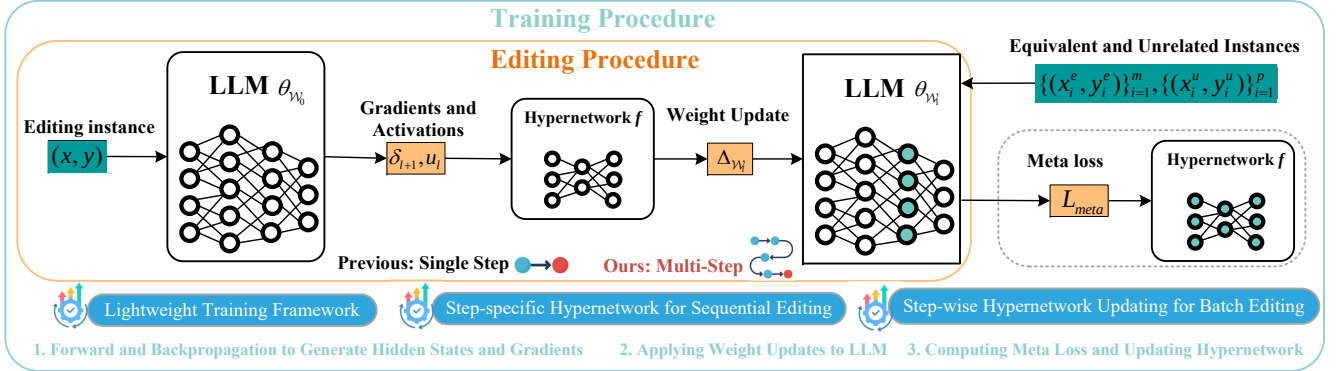


Figure 1: Overview of EMSEdit. Existing MLME methods perform model editing in a single step (i.e., one-step backpropagation), whereas EMSEdit conducts editing through multi-step updates, similar to multi-step backpropagation.

representative locate-then-edit method) is approximately $27.8 \times$ slower than RLEdit [16], a representative MLME method. MLME approaches exhibit promising performance both in terms of effectiveness and efficiency.

Despite the progress made in MLME, we observe that it still has a couple of limitations. First, current MLME methods perform well when abundant training data is available, but their performance is suboptimal under low-data scenarios (Section 4.1). This highlights a data efficiency limitation in existing MLME methods. Second, the training efficiency of MLME is bottlenecked by the computation of KL divergence loss (Section 4.2), which requires the reference probability distribution from the original model, leading to double forward passes per iteration.

To tackle the above issues, we propose **Efficient Multi-Step Edit (EMSEdit)**, which performs model editing through multi-step edits on each editing sample, leveraging multi-step backpropagation (MSBP) to sufficiently learn editing patterns from the limited training data. This enables more effective editing with limited data, thereby addressing the first issue. To improve training efficiency, EMSEdit replaces the KL loss with an l_2 regularization on the weight update to preserve the original model’s knowledge, a design we term the lightweight training framework, thus alleviating the second issue. Furthermore, EMSEdit introduces a step-specific hypernetwork for sequential editing and a step-wise hypernetwork updating mechanism for batch editing, balancing editing effectiveness and efficiency. The comparison between EMSEdit and existing MLME methods is illustrated in Figure 1.

We evaluate EMSEdit on both batch and sequential editing tasks using GPT-J (6B) [28], LLaMA-3 (8B) [2], and Gemma-2 (9B) [26] on the ZsRE [39] and CounterFact [19] datasets. Experimental results demonstrate that **EMSEdit surpasses state-of-the-art methods in both sequential and batch editing**. We further validate that EMSEdit achieves superior performance on the multi-hop reasoning editing task, and that **incorporating MSBP consistently enhances the editing performance of existing MLME baselines**. In addition, we validate the effectiveness of the components introduced in EMSEdit through ablation studies and further demonstrate its data and memory efficiency through additional analyses. In summary, our contributions are as follows:

- We reveal that current MLME methods face limitations in both data and training efficiency. Their performance is sub-optimal under low-data scenarios and training efficiency is bottlenecked by the computation of KL loss.
- We propose to use MSBP and norm constraints to address the limitations of existing MLME methods. We thus introduce EMSEdit, a novel MLME approach that effectively balances editing performance and efficiency through the integration of MSBP and three key designs.
- We demonstrate the effectiveness of EMSEdit and MSBP through sequential and batch editing experiments on three LLMs and two datasets. Our additional experiments and analyses respectively demonstrate that EMSEdit remains effective on complex multi-hop reasoning editing tasks and exhibits superior data and memory efficiency.

2 Related Work

Model editing has recently emerged as a promising technique for updating the knowledge of large language models (LLMs) [35]. These approaches can be broadly categorized into two types based on whether they modify the parameters of LLMs: **parameter-altering** and **parameter-preserving** methods [34].

Parameter-preserving methods store editing information externally—either by retrieving relevant knowledge from external sources [38], introducing auxiliary modules outside the model [22], or adding new internal parameters [12, 29]. These approaches have shown promising results in model editing tasks while mitigating the risks of altering the model’s original behavior. However, the external modules often increase system complexity and computational overhead, limiting their practicality in real-world applications.

In contrast, **parameter-altering** methods produce edited models that differ from the original ones only in their parameters [19, 21, 39]. This design makes them inherently more lightweight and flexible, allowing the edited models to be further edited or adapted to downstream tasks seamlessly. Such methods typically perform editing by modifying a small subset of the LLM’s parameters. For example, locate-then-edit methods apply least-squares updates to key layers believed to store factual knowledge [8, 15, 20],

achieving promising editing results. However, these methods often suffer from limited editing efficiency.

Meta-learning-based model editing (MLME) represents a class of approaches that are competitive in both effectiveness and efficiency [16, 25]. These methods train a lightweight hypernetwork to transform single-step fine-tuning gradients and internal representations into effective weight updates [21]. In this paper, we propose a novel MLME method, EMSEdit. Unlike existing MLME methods, EMSEdit leverages MSBP to make fuller use of the editing data, thereby enabling more expressive and effective model editing.

3 Preliminaries

3.1 Model Editing Problem

Model editing aims to efficiently modify a LLM θ to achieve specific editing goals without affecting its other capabilities. We define an editing sample as

$$T = (x, y) \cup \{(x_i^e, y_i^e)\}_{i=1}^m \cup \{(x_i^u, y_i^u)\}_{i=1}^p \quad (1)$$

where (x, y) is the editing instance used to measure the **efficacy** of model editing methods; $(x_i^e, y_i^e)_{i=1}^m$ are examples that are semantically equivalent to (x, y) and are used to evaluate the **generalization** ability; and $(x_i^u, y_i^u)_{i=1}^p$ are semantically unrelated examples used to assess the **specificity** of the model editing methods. Additionally, to assess whether the model retains its original capabilities after editing, existing works evaluate the performance of edited models on downstream natural language processing tasks (e.g., GLUE [27]). Model editing methods can be classified into three scenarios: **sequential editing**, **batch editing**, and **sequential batch editing** [18]. Given n editing samples $\{T_1, T_2, \dots, T_n\}$ to be edited, the three scenarios can be described as follows: **a) Sequential Editing**: This scenario continuously edits a single piece of knowledge, processing the n editing samples one by one; **b) Batch Editing**: This scenario performs a single edit operation on all n samples simultaneously, effectively editing multiple pieces of knowledge at once; **c) Sequential Batch Editing**: This scenario divides the n samples into batches of size b and performs $\lceil \frac{n}{b} \rceil$ sequential editing steps, each handling a batch.

After completing the editing process, the performance of the editing method is evaluated by measuring the success rate of the edited model across the n editing samples. The ultimate goal of all three scenarios is to effectively write the n knowledge samples into θ . However, their applicable scenarios, approaches to achieving the goal, and implementation challenges differ significantly.

3.2 Meta Learning Based Model Editing

Meta-learning-based model editing trains a hypernetwork to convert standard fine-tuning gradients into weight updates, reducing the computational burden of direct weight adjustment [16, 21, 25].

Editing Procedure: We denote the weight of θ as: $\mathcal{W} = \{W_l : l \in \mathcal{L}\}$ where W_l is the weight of linear layer l and $\mathcal{L}$ is the set of all linear layers. MEND [21] decomposes the single BP gradient ∇_{W_l} of the weight $W_l \in \mathbb{R}^{d' \times d}$ in layer l to be updated into a rank-1 product: $\nabla_{W_l} = \delta_{l+1} u_l^T$, where $\delta_{l+1} \in \mathbb{R}^{d' \times 1}$ is the gradient of loss L with respect to the preactivations to layer $l+1$ and $u_l \in \mathbb{R}^{d \times 1}$ are the inputs to layer l . The weight update $\Delta_{W_l} \in \mathbb{R}^{d' \times d}$ is obtained as the product of the *pseudogradient* and *pseudooactivations* transformed

by a hypernetwork:

$$\Delta_{W_l} = \tilde{\delta}_{l+1} \tilde{u}_l^T, \text{ where } \tilde{\delta}_{l+1}, \tilde{u}_l^T = f(\delta_{l+1}, u_l^T) \quad (2)$$

where $f : \mathbb{R}^d \times \mathbb{R}^{d'} \rightarrow \mathbb{R}^d \times \mathbb{R}^{d'}$ is the hypernetwork which transforms the gradient and inputs to *pseudogradient* $\tilde{\delta}_{l+1}$ and *pseudooactivations* $\tilde{u}_l^T$. MALMEN [25] improves the above process to accommodate massive batch editing scenarios by formulating a least squares problem that aggregates the weight updates corresponding to each editing sample into a single weight update. The weight update Δ_{W_l} of MALMEN is:

$$\Delta_{W_l} = D_l U_l^T (U_l U_l^T + \lambda I)^{-1} \quad (3)$$

by solving: $\min_{\Delta_{W_l} \in \mathbb{R}^{d' \times d}} \|\Delta_{W_l} U_l - D_l\|_2^2 + \lambda \|\Delta_{W_l}\|_2^2$

where $D_l = [\dots, d_{l,k}, \dots] = [\dots, \tilde{\delta}_{l+1,k} \tilde{u}_{l,k}^T, \dots] \in \mathbb{R}^{d' \times b}$ and $U_l = [\dots, u_{l,k}, \dots] \in \mathbb{R}^{d \times b}$, with k representing the k -th editing sample in a batch of size b .

Training Procedure: The purpose of this process is to obtain a hypernetwork f that can transform gradients and activations into weight updates capable of achieving the model editing objective. The goal of model editing is to perform the edit while maintaining good generalization and specificity, which corresponds to the following two loss functions:

$$L_e = -\log \theta_{W_1}(y^e | x^e), L_{loc} = \text{KL}[\theta_{W_0}(\cdot | x^u) || \theta_{W_1}(\cdot | x^u)] \quad (4)$$

where θ_{W_0} and θ_{W_1} denote the LLM with the original weights and the updated weights after one edit, respectively. The meta loss for training f is the sum of L_e and L_{loc} : $L_{meta} = L_e + \lambda_{loc} L_{loc}$ where λ_{loc} is a hyperparameter that balances generalization and specificity.

Based on the meta loss, different methods adopt various implementation details. MEND back-propagates L_{meta} into f via LLM θ [21], while MALMEN back-propagates on θ and f separately to avoid high memory costs [25]. RLEdit introduces a reinforcement learning framework into the above process, thus enabling the hypernetwork to perform sequential model editing in θ [16]. Specifically, RLEdit optimizes the following reward to obtain the optimal parameters of f :

$$J = \sum_{i=1}^n \gamma^i (L_{meta_i} + L_{back_i} + \eta \|\Delta_{W_i}\|^2) \quad (5)$$

where the term $\eta \|\Delta_{W_i}\|^2$ is the regularization term, with coefficient η , that accounts for the weight updates Δ_{W_i} at edit step i . γ^i is the discount factor. The term L_{back_i} is the memory backtracking loss. RLEdit computes the meta loss of the current editing sample T_i and the meta loss of previous q editing samples $\{T_{i-1}, \dots, T_{i-q}\}$ on $\theta_{W_{i-1}}$ at edit step i . The backtracking loss is defined as:

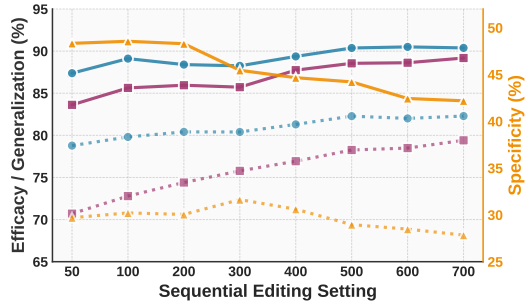
$$L_{back_i} = \sum_{j=i-q}^{i-1} \mu^{i-j} (L_{e_j, W_{i-1}} + \lambda_{loc} L_{loc_j, W_{i-1}}) \quad (6)$$

where μ is a decay factor.

4 Limitations of MLME

In this study, we use two state-of-the-art MLME methods as research subjects and baselines: RLEdit [16] for sequential editing and MALMEN [25] for batch editing.

Performance Variation with Sequential Editing Settings



Performance Variation with Training Steps in Batch Editing

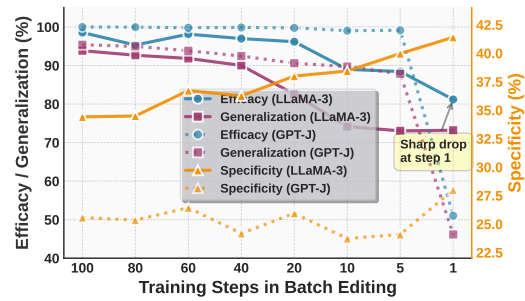


Figure 2: Performance Variation of MLME in Sequential and Batch Editing. The batch size for sequential editing is 20.

4.1 Data Efficiency Limitation

We observe that MLME’s performance follows a saturating growth trend with respect to the amount of training data it receives. In Figure 2, we show how MLME’s editing performance varies with the number of sequential edits and training steps in batch edits.

In the sequential editing setting¹, the overall editing performance increases as the number of sequential edits grows. This result is somewhat counterintuitive, as one might expect sequential model editing to become more challenging with a greater number of edits. We hypothesize that longer editing sequences expose the model to more training instances, thereby enhancing its performance.

To test this hypothesis, we conduct batch editing experiments involving 1,000 batch edits, varying the number of training steps to simulate different amounts of training data. At each step, the number of training samples is equal to the batch size, and the training data differed across steps. As shown in Figure 2 (bottom), editing performance decreases as the number of training steps is reduced. Especially, when the number of training steps is 1—meaning the amount of training data is equal to the batch size—the editing performance of MLME drops significantly. This further supports our hypothesis, leading to the conclusion that **the performance of MLME improves with increased training data, and that when training data is scarce, MLME suffers a substantial performance degradation**. Given that training data is often scarce or hard to acquire in real-world applications [14], the **current**

¹In our sequential editing experiments with RLEdit, the number of training samples is matched to the number of sequential edits, following the same setup as in [16].

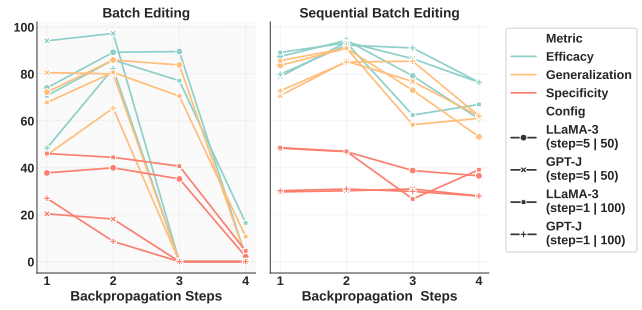


Figure 3: Performance Variation of MLME in Sequential and Batch Editing Across Different BP Steps on ZsRE Dataset. (step=5 | 50) means the training steps is 5 in batch editing and sequence number is 50 in sequential editing.

design of MLME yields suboptimal editing performance in low-resource settings, thereby constraining its practical utility.

4.1.1 Solutions to Mitigate the Data Efficiency Limitation. The most direct way to mitigate the limitations of MLME is to increase the amount of training data. However, this approach is difficult to implement in data-scarce scenarios. Another option is to modify the training strategy of MLME. We observe that the current MLME performs model editing using only a single-step gradient, which is equivalent to performing single backpropagation. As a result, MLME may not fully capture the editing patterns in the training process. To verify this, we modify MLME to adopt a multi-step gradient-based approach, where multi-step backpropagation (MSBP) is performed on a batch of data to achieve model editing.

We show the performance variation of MLME in both batch and sequential batch editing settings across different numbers of BP steps in Figure 3. In the batch editing setting (left), we set the number of batches as 8,192. We limit the number of training steps to simulate the scarcity of training data in real-world scenarios. The efficacy and generalization of MLME generally exhibit a trend of first increasing and then decreasing, while specificity consistently decreases as the number of backpropagation (BP) steps increases. In the sequential batch editing (right figure in Figure 3, batch=30), the trends observed here are similar to those in batch editing, but the best editing performance is achieved when the step size is 2. This result is intuitive: more BP steps mean more updates, and performing multiple updates on a single batch can lead to overfitting to that batch, thereby increasing efficacy and generalization but reducing specificity. Moreover, when the number of BP steps exceeds a certain threshold, the editing performance drops sharply, which may be due to excessive updates causing the editing collapse. This suggests that **MSBP can indeed achieve better editing performance**, and the choice of step size is a critical factor, with a step size of 2 yielding the overall best results.

4.2 Training Efficiency Limitation of MLME

Although MLME demonstrates strong editing efficiency, its training efficiency remains suboptimal. To investigate this issue, we conduct an in-depth analysis of the training time distribution for two representative MLME methods, MALMEN and RLEdit, under both

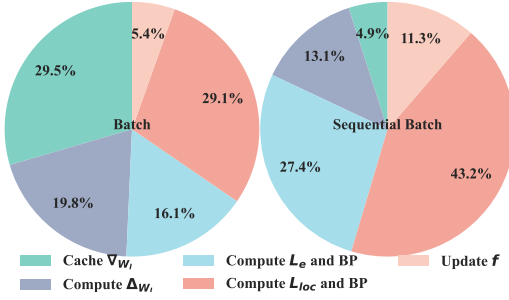


Figure 4: MLME's Time Distribution.

batch editing and sequential batch editing settings. Specifically, we perform batch editing with a batch size of 8,192 and sequential batch editing with 30×200 updates using MALMEN and RLEdit on the ZsRE and CounterFact datasets, employing GPT-J and LLaMA-3 (8B) as the underlying models. We measure the average time consumption of the five steps in the MLME training process: 1. Caching ∇_{W_l} ; 2. Computing Δ_{W_l} ; 3. Computing L_e and BP; 4. Computing L_{loc} and BP; 5. Updating f .

Figure 4 shows the time distribution of MLME. In both the Batch and Sequential Batch experiments, Step 4 (computing L_{loc} and BP) accounts for a significant portion of the total training time—29.1% and 43.2%, respectively. This step is designed to protect the model's original knowledge. This overhead is expected because calculating the KL loss requires two forward passes through the LLMs (one for the original model output and one for the edited model output), leading to increased computational cost. Although Step 1 (Cache ∇_{W_l}) consumes more time than Step 4 in the Batch Editing setting, it involves collecting the original fine-tuning gradients and internal representations, which is a fundamental step in MLME. Notably, its time cost is the lowest in Sequential Batch Editing, indicating that it is not the primary bottleneck.

From the above analysis, we can conclude that **the training bottleneck of MLME lies in Step 4: Compute L_{loc} and BP**. Therefore, we need to find an alternative way to efficiently preserve the model's original knowledge. To achieve this, we introduce the l_2 -norm constraint as an alternative in Section 5.1, and demonstrate in the batch editing and sequential editing experiments (Section 6) that this alternative can achieve the aforementioned goal.

5 Method

Following the same editing pipeline as MLME, **Efficient Multi-Step Edit (EMSEdit)** consists of an **Editing Procedure** and a **Training Procedure**. However, EMSEdit incorporates *MSBP-based editing*, *step-specific hypernetwork for sequential editing*, *step-wise hypernetwork updating for batch editing*, and an *efficient and lightweight training process*. We describe them in detail below.

5.1 Lightweight Training

As shown in Section 3.2, previous MLME methods typically use the weighted sum of the generalization loss L_e and the specificity loss L_{loc} as the meta-loss for training the hypernetwork. While this meta-loss effectively balances the trade-off between editing new knowledge and preserving original knowledge, we show in Section

Algorithm 1 Hypernetwork Training for Sequential Editing

```

1: Input: LLM  $\theta_{W_0}$ , hyperparameters  $S$  and  $\eta$ , set of hypernetworks  $\mathcal{F} = \{f_1, \dots, f_S\}$ 
2: Output: Optimized hypernetworks  $\mathcal{F}$ 
3: Sample  $(T_i = \{x_i, y_i, x_i^e, y_i^e\})_{i=1}^n$ 
4: // Efficient and lightweight training process
5: for  $t = 1$  to  $n$  do
6:   // MSBP
7:   for  $s = 1$  to  $S$  do
8:      $L \leftarrow -\log \theta_{W_{t-1}^{s-1}}(y_t | x_t)$ 
9:     Back-propagate  $L$  and cache  $\nabla_{W_{t-1}}$ 
10:    // Step-specific hypernetwork
11:     $\Delta_{s-1, W_{t-1}} \leftarrow f_{s-1}(\nabla_{W_{t-1}})$ 
12:     $W_t^s \leftarrow W_{t-1}^{s-1} + \Delta_{s-1, W_{t-1}}$ 
13:     $L_{cons} \leftarrow \|W_t^s - W_0\|^2$ 
14:     $L_{e,i} \leftarrow -\log \theta_{W_t}(y_i^e | x_i^e)$ 
15:     $L_i \leftarrow L_{e,i} + \eta L_{cons}$ 
16:    Back-propagate  $L_i$ 
17:    Update  $\mathcal{F}$ 
18: return  $\mathcal{F}$ 

```

Algorithm 2 Hypernetwork Training for Batch Editing

```

1: Input: LLM  $\theta_{W_0}$ , hyperparameters  $S$  and  $\eta$ , a hypernetwork  $f$ 
2: Output: An optimized hypernetwork  $f$ 
3: Sample  $(T_i = \{x_i, y_i, x_i^e, y_i^e\})_{i=1}^n$ 
4: // Efficient and lightweight training process
5: for  $t = 1$  to  $n$  do
6:   // MSBP
7:   for  $s = 1$  to  $S$  do
8:     // Step-wise hypernetwork updating
9:      $L \leftarrow -\log \theta_{W_{t-1}^{s-1}}(y_t | x_t)$ 
10:    Back-propagate  $L$  and cache  $\nabla_{W_{t-1}}$ 
11:     $\Delta_{s-1, W_{t-1}} \leftarrow f(\nabla_{W_{t-1}})$ 
12:     $W_t^s \leftarrow W_{t-1}^{s-1} + \Delta_{s-1, W_{t-1}}$ 
13:     $L_{cons} \leftarrow \|W_t^s - W_0\|^2$ 
14:     $L_{e,i} \leftarrow -\log \theta_{W_t}(y_i^e | x_i^e)$ 
15:     $L_i \leftarrow L_{e,i} + \eta L_{cons}$ 
16:    Back-propagate  $L_i$ 
17:    Update  $f$ 
18: return  $f$ 

```

4.2 that computing the loss L_{loc} leads to inefficient training. To improve training efficiency while preserving the original model's knowledge as much as possible, inspired by RLEdit, we replace the specificity loss with an l_2 -norm constraint loss L_{cons} on the weight updates. Therefore, the final meta-loss of EMSEdit is formulated as:

$$L_{meta} = L_e + \eta L_{cons} = L_e + \eta \sum_{s=0}^S \|\Delta_{s, W_t}\|^2 \quad (7)$$

5.2 Step-specific Hypernetwork

EMSEdit achieves a multi-step optimization process similar to traditional deep learning training through MSBP. In sequential batch editing, the hypernetwork must model both **inter-step variance** (across BP steps) and **inter-batch variance** (across batches), which is challenging for a single small hypernetwork. To address this, we adopt step-specific hypernetworks that specialize in capturing

Method	GPT-J (6B)						LLaMA-3 (8B)						Gemma-2 (9B)					
	Eff.↑	Gen.↑	Spe.↑	Avg.↑	Time (s) / Edit		Eff.↑	Gen.↑	Spe.↑	Avg.↑	Time (s) / Edit		Eff.↑	Gen.↑	Spe.↑	Avg.↑	Time (s) / Edit	
					Train↓	Val↓					Train↓	Val↓					Train↓	Val↓
ZsRE-Sequential Batch Editing-30×600																		
FT	50.79	47.98	29.47	42.75	-	0.14	69.67	62.27	32.22	54.72	-	0.16	7.06	6.38	9.96	7.80	-	0.28
MEND	0.60	0.62	0.52	0.58	0.10	0.04	0.21	0.15	0.06	0.14	0.15	0.06	21.43	19.88	14.05	18.45	0.10	0.07
RLEdit	83.76	81.47	29.13	64.79	0.50	0.05	90.16	88.78	42.04	73.66	0.67	0.09	84.60	79.66	35.67	66.64	0.67	0.11
UltraEdit	65.53	61.59	31.34	52.82	-	0.04	87.41	84.29	49.94	73.88	-	0.06	83.26	77.17	50.03	70.15	-	0.08
EMSEdit	93.61	90.34	30.64	71.53	0.17	0.16	95.40	93.91	42.18	77.16	0.19	0.11	89.55	85.11	36.15	70.27	0.25	0.16
CounterFact-Sequential Batch Editing-20×400																		
FT	40.72	33.72	62.29	45.58	-	0.16	83.70	52.44	51.63	62.59	-	0.18	48.39	36.27	51.24	45.30	-	0.31
MEND	42.12	42.05	57.88	47.35	0.08	0.08	48.67	49.44	50.32	49.48	0.14	0.07	25.14	26.23	72.70	41.36	0.22	0.11
RLEdit	61.34	45.76	54.06	53.72	0.53	0.15	63.95	57.56	41.72	54.41	0.54	0.07	60.31	50.16	48.34	52.94	0.65	0.08
UltraEdit	43.23	32.81	58.85	44.96	-	0.06	63.55	48.37	43.87	51.93	-	0.06	66.43	49.97	49.97	55.46	-	0.07
EMSEdit	94.43	65.56	44.48	66.82	0.26	0.12	94.24	79.49	40.56	71.43	0.30	0.14	89.74	70.12	42.37	67.41	0.28	0.16
ZsRE-Batch Editing-16384																		
FT	26.90	26.20	27.31	26.80	-	0.14	38.35	36.77	32.11	35.74	-	0.16	3.21	3.01	6.37	4.20	-	0.27
MEND	0.01	0.01	0.00	0.01	0.07	0.02	0.01	0.02	0.02	0.02	0.11	0.04	23.58	22.68	13.18	19.81	0.15	0.06
MALMEN	46.05	44.14	27.29	39.16	0.05	0.02	69.28	66.59	45.46	60.44	0.07	0.04	78.76	71.85	42.18	64.26	0.13	0.05
EMSEdit	82.66	72.73	25.81	60.40	0.07	0.04	84.15	80.94	49.41	71.50	0.11	0.07	96.57	90.34	39.31	75.41	0.19	0.09
CounterFact-Batch Editing-8192																		
FT	15.17	17.66	83.22	38.68	-	0.14	7.07	9.63	89.37	35.36	-	0.15	49.68	31.12	49.31	43.37	-	0.28
MEND	44.76	46.60	54.53	48.63	0.09	0.03	43.73	45.11	55.80	48.21	0.10	0.03	15.03	19.28	80.80	38.37	0.13	0.04
MALMEN	66.92	29.64	75.86	57.47	0.04	0.03	51.06	29.63	78.42	53.04	0.06	0.03	94.57	58.86	62.46	71.96	0.09	0.04
EMSEdit	99.82	50.07	61.44	70.44	0.05	0.04	97.96	60.42	57.83	72.07	0.08	0.05	99.06	73.24	50.03	74.11	0.10	0.06

Table 1: Comparison of Editing Methods across GPT-J (6B), Llama-3 (8B), and Gemma2 (9B) on ZsRE and CounterFact Datasets in Sequential Editing Task. Eff.: Efficacy, Gen.: Generalization, Spe.: Specificity. A setting of 30×600 indicates performing 600 sequential batch edits, each with a batch size of 30. The number following batch editing denotes the batch size.

inter-batch differences. Specifically, EMSEdit performs editing over S BP steps, where the s -th step is handled by a hypernetwork f_s . These hypernetworks form a set $\mathcal{F} = f_1, \dots, f_S$. The corresponding training procedure is shown in Algorithm 1.

Each hypernetwork shares the same architecture as prior MLME methods [21, 25]. As gradients typically diminish over steps, learning the gradient-to-pseudo-gradient transformation becomes easier in later stages. Accordingly, we apply a rank decay strategy to reduce the editing burden, defining the rank at step s as $r_s = r \mid s$, where r is a predefined rank.

5.3 Step-wise Hypernetwork Updating

Batch editing performs a single edit with a large batch size, where the primary source of variance arises from inter-step variance. In this setting, a single hypernetwork is sufficient and reduces memory overhead compared to maintaining multiple networks. Accordingly, batch editing training employs a single hypernetwork f to learn multi-step gradient transformations, as it better captures underlying transformation patterns. To further reduce memory consumption caused by gradient accumulation, especially with large batch sizes, we adopt step-wise hypernetwork updates, updating the hypernetwork at each backpropagation step. The pseudo-code of EMSEdit hypernetwork training for batch editing is shown in Algorithm 2.

6 Experiments

6.1 Experimental Settings

LLMs: GPT-J (6B) [28], LLaMA-3 (8B) [2], and Gemma-2 (9B) [26].

DataSets & Metrics We consider two common datasets in model editing: CounterFact [19] and ZsRE [1]. In line with prior work [16, 25], we evaluate editing performance using three metrics: Efficacy, Generalization, and Specificity. Details are shown in Appendix D.

Baselines Our work primarily addresses the limitations of MLME. For sequential editing, the baselines include FT [35], MEND [21], RLEdit [16], and UltraEdit [11]. For batch editing, we use FT [35], MEND [21], and MALMEN [25] as our baselines.

Experimental Details These details are shown in Appendix D.

6.2 Main Results

6.2.1 Results on Sequential Batch Editing. We evaluate EMSEdit on sequential batch editing using GPT-J (6B), LLaMA-3 (8B), and Gemma-2 (9B) on the ZsRE and CounterFact datasets. As shown in the upper part of Table 1, EMSEdit achieves the best overall performance across all models and datasets, with substantial gains in Efficacy and Generalization over existing MLME methods such as RLEdit and UltraEdit. For example, on ZsRE, EMSEdit reaches 93.61/95.40/89.55 in Efficacy, outperforming RLEdit by a large margin. Although its Specificity is slightly lower than UltraEdit in some cases, EMSEdit attains the highest Average score, indicating a strong balance between editing effectiveness and model stability. These results demonstrate that MSBP enables effective learning from limited editing data.

In terms of efficiency, EMSEdit maintains competitive editing speed, with training and validation time per edit comparable to or lower than other MLME methods and significantly faster than

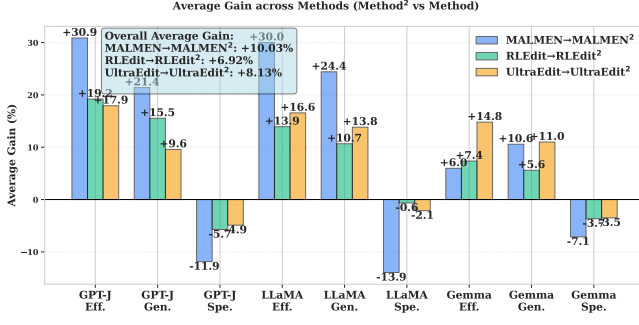


Figure 5: Performance Gains of MSBP over Existing Methods.

fine-tuning. Overall, EMSEdit achieves a superior trade-off between performance and efficiency under sequential editing.

6.2.2 Results on Batch Editing. In the batch editing scenarios on ZsRE and CounterFact, EMSEdit continues to deliver the strongest performance among all baselines. Across all three models, EMSEdit achieves the highest scores in Efficacy, Generalization, and Average, demonstrating remarkable robustness under large-scale editing. For example, on ZsRE (Gemma-2), EMSEdit attains an Average of 75.41, surpassing MALMEN (64.26) by a significant margin. Similarly, on CounterFact, EMSEdit obtains an Average of 70.44, 72.07, and 74.11 on GPT-J, LLaMA-3, and Gemma-2, respectively—establishing new state-of-the-art results in batch editing. Although EMSEdit’s Specificity is slightly below MALMEN in isolated cases, its overall balance across metrics is more consistent, suggesting stable model behavior even when multiple edits are performed simultaneously.

From an efficiency perspective, EMSEdit remains lightweight and scalable. Its per-edit training and validation time (e.g., 0.07 s / 0.04 s on GPT-J for ZsRE) is on par with MALMEN while achieving much higher performance, showing that EMSEdit’s design yields both computational efficiency and superior editing quality.

6.2.3 Performance Gains of MSBP over Existing Methods. To evaluate the effectiveness of MSBP, we apply a two-step BP mechanism to existing MLME methods, including MALMEN, RLEdit, and UltraEdit, and evaluate them on sequential batch and batch editing tasks. Figure 5 shows the average performance gains across different LLMs, where MALMEN², RLEdit², and UltraEdit² denote the enhanced versions. MSBP consistently improves Efficacy and Generalization (e.g., +20.55% for MALMEN), while causing moderate declines in Specificity (e.g., -3.3% for RLEdit), indicating increased interference with unedited knowledge due to overfitting. Overall, MSBP yields clear average performance gains of 10.03%, 6.92%, and 8.13% for MALMEN, RLEdit, and UltraEdit, respectively, demonstrating its effectiveness across methods and models. Additional results under different sequential and batch settings are provided in Appendix A.

6.2.4 Comparison Results on multi-hop reasoning dataset. To evaluate the performance of EMSEdit on multi-hop reasoning model editing, we conduct experiments on the RippleEdit [5] dataset using LLaMA-3 (8B), GPT-J (6B), and Gemma-2 (9B). We follow [36] to split the dataset into training and test sets, and maintain the same experimental settings as in the main experiments. We report the

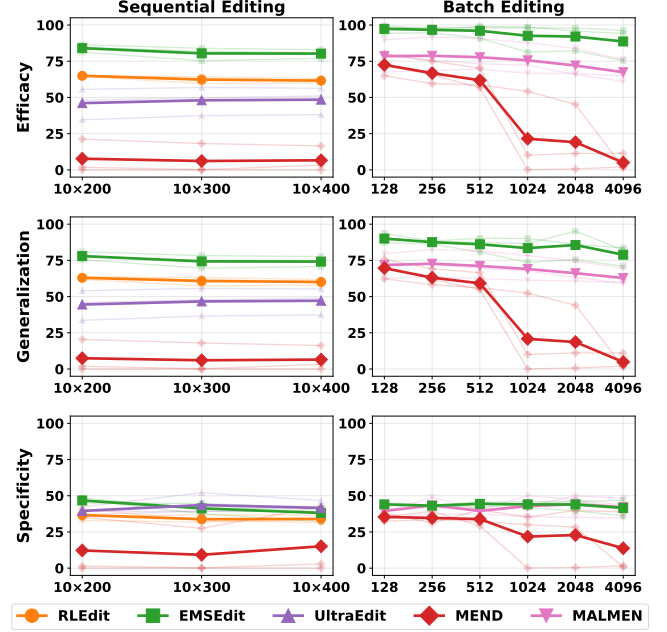


Figure 6: Average Editing Performance Across LLMs. Individual method performance is shown in lighter shades.

results in Figure 6. As shown, EMSEdit consistently demonstrates superior performance in both sequential and batch editing scenarios across different models. It not only achieves higher editing efficacy and generalization but also maintains strong specificity, confirming its robustness and scalability.

6.3 Ablation Study

We verify the number of BP steps selected by EMSEdit, as well as the effectiveness of *Step-specific Hypernetwork* (corresponding to *identical hypernetwork*) and *rank decay* (corresponding to *consistent rank*) for sequential editing, and *Step-wise Hypernetwork Updating* (corresponding to *No step-wise*) and *Identical Hypernetwork* (corresponding to *diff hypernetwork*) for batch editing. We conduct 8192-sized batch editing and 30×400 sequential batch editing on the LLaMA-3 (8B) model using the ZsRE dataset. The results are shown in Table 2, with EMSEdit results highlighted in cyan.

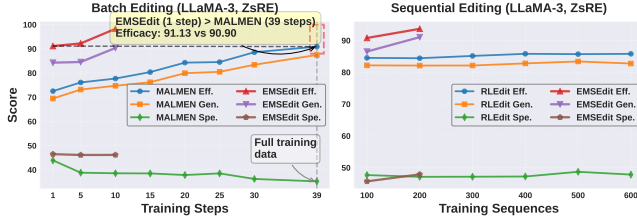
Regarding the number of BP steps, although a BP step of 3 and 4 achieves better editing performance in batch editing, its training and validation time nearly doubles. Choosing 2 steps achieves a better balance between performance and efficiency. For sequential batch editing, using 2 BP steps yields the best balance between performance and efficiency.

In terms of the hypernetwork design, the results indicate that using different hypernetworks in batch editing and the same hypernetwork in sequential batch editing both lead to worse editing performance. This may be due to the differing nature of batch and sequential editing tasks. Additionally, maintaining a consistent hypernetwork rank in sequential batch editing does not yield

Table 2: Ablation Study.

Bath Size: 8192					
Metrics	Eff.	Gen.	Spe.	Train (s) / Edit	Val (s) / Edit
BP Step 1	70.18	67.22	46.01	0.05	0.04
BP Step 2 (EMSEdit)	85.91	80.64	44.44	0.11	0.07
BP Step 3	91.19	84.50	47.60	0.15	0.11
BP Step 4	94.55	86.91	49.01	0.20	0.14
Diff HyperNetwork	83.57	78.13	46.88	0.12	0.08
No Step-wise	Out of Memory				
Sequence Length×Bath Size: 30×400					
BP Step 1	89.31	87.57	43.47	0.11	0.06
BP Step 2 (EMSEdit)	95.61	94.09	44.94	0.18	0.11
BP Step 3	88.08	85.86	36.88	0.29	0.19
BP Step 4	55.19	49.39	16.42	0.56	0.21
Identical HyperNetwork	93.57	91.83	38.66	0.35	0.12
Consistent Rank	94.52	92.53	44.02	0.20	0.12

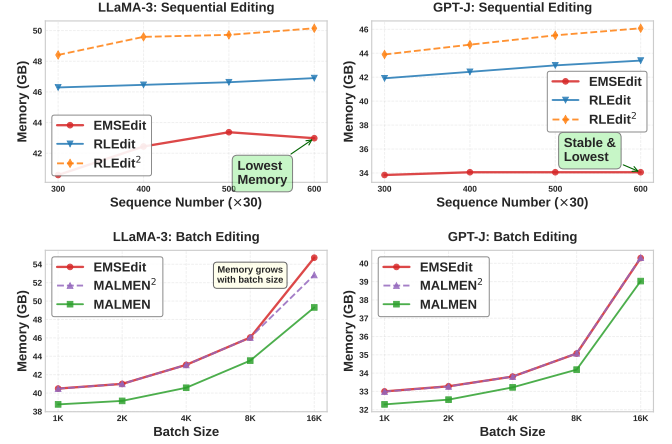
better editing results. Furthermore, in batch editing, not using step-wise updating requires substantial memory for gradient accumulation—we also experimented with a batch size of 1024 and still encountered out-of-memory errors.

**Figure 7: Data Efficiency of EMSEdit.**

6.4 Data Efficiency of EMSEdit

We conduct sequential and batch editing experiments on ZsRE with LLaMA-3 to evaluate the data efficiency of EMSEdit. For sequential editing, we use a batch size of 4 and vary the number of training sequences from 100 to 600, with 600 validation sequences. For batch editing, we fix the validation batch size to 4096 and vary the training steps from 1 to 39 (full data). Other settings follow the main experiments, and results are shown in Fig. 7.

In batch editing, MALMEN’s performance improves with increasing training steps and reaches its best performance only after all training data have been used. In contrast, EMSEdit surpasses MALMEN’s best performance with only a single training step and continues to improve with additional steps. In sequential editing, RLEdit shows a modest performance gain as the training sequence length increases, but plateaus at a length of 400 with a maximum efficacy of 85.80. By comparison, EMSEdit exceeds RLEdit’s best performance with a sequence length of just 100. These results demonstrate that EMSEdit is substantially more data-efficient than both MALMEN and RLEdit, achieving superior performance with far less data, even when the baselines are trained on much larger datasets. This suggests that MSBP improves the utilization efficiency of editing data, analogous to conventional deep learning training where

**Figure 8: Memory Efficiency of EMSEdit.**

repeated iterations over the same dataset enhance learning effectiveness. Consistent with this interpretation, Appendix B shows that under MSBP, the loss in EMSEdit progressively decreases as the number of backpropagation steps increases.

6.5 Memory Efficiency of EMSEdit

Figure 8 reports the memory usage of EMSEdit, MALMEN, and RLEdit in our main experiments. The results show that EMSEdit exhibits high memory efficiency, particularly in sequential editing, where it outperforms MALMEN and RLEdit by a wide margin. In batch editing, EMSEdit and MALMEN² display nearly identical memory profiles, indicating comparable efficiency, although EMSEdit incurs slightly higher memory usage at larger batch sizes. Overall, EMSEdit demonstrates good scalability and stable memory behavior across all settings, making it a practical choice for memory-constrained or resource-limited scenarios.

7 Conclusion

We propose EMSEdit, a new MLME method that improves data and training efficiency. By leveraging multi-step backpropagation (MSBP), EMSEdit performs effective editing in low-data regimes, while replacing KL divergence with l_2 regularization reduces computational overhead without degrading performance. Our step-specific and step-wise hypernetwork designs further improve efficiency and performance across sequential and batch settings. Experiments on GPT-J, LLaMA-3, and Gemma-2 over ZsRE and CounterFact show that EMSEdit achieves state-of-the-art results and demonstrate the broader potential of MSBP for model editing.

Acknowledgements

This work was supported by the National Key Research and Development Program of China under Grant 2024YFB4506200, the Science and Technology Innovation Program of Hunan Province under Grant 2024RC1048, the National Key Laboratory Foundation Project under Grant 2024-KJWPD-14, the National Natural Science Foundation of China under Grant 62402506, and the Innovation Research Foundation of (NUDT) (HQKYZH2025KD004).

References

- [1] 2017. Zero-shot relation extraction via reading comprehension. In *CoNLL 2017 - 21st Conference on Computational Natural Language Learning, Proceedings (CoNLL 2017 - 21st Conference on Computational Natural Language Learning, Proceedings)*. Association for Computational Linguistics (ACL), 333–342. doi:10.18653/v1/k17-1034
- [2] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [3] Luisa Bentivogli, Peter Clark, Ido Dagan, and Danilo Giampiccolo. 2009. The Fifth PASCAL Recognizing Textual Entailment Challenge. *TAC* 7, 8 (2009), 1.
- [4] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*. PMLR, 2397–2430.
- [5] Roi Cohen, Eden Biran, Ori Yoran, Amir Globerson, and Mor Geva. 2024. Evaluating the ripple effects of knowledge editing in language models. *Transactions of the Association for Computational Linguistics* 12 (2024), 283–298.
- [6] Bill Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Third international workshop on paraphrasing (IWP2005)*.
- [7] Yanai Elazar, Nora Kassner, Shauli Ravfogel, Abhilasha Ravichander, Eduard Hovy, Hinrich Schütze, and Yoav Goldberg. 2021. Measuring and improving consistency in pretrained language models. *Transactions of the Association for Computational Linguistics* 9 (2021), 1012–1031.
- [8] Junfeng Fang, Houcheng Jiang, Kun Wang, Yunshan Ma, Xiang Wang, Xiangnan He, and Tat-seng Chua. 2024. Alphaedit: Null-space constrained knowledge editing for language models. *arXiv preprint arXiv:2410.02355* (2024).
- [9] Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. 2023. Dissecting Recall of Factual Associations in Auto-Regressive Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 12216–12235. doi:10.18653/v1/2023.emnlp-main.751
- [10] Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. 2021. Transformer Feed-Forward Layers Are Key-Value Memories. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.). Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 5484–5495. doi:10.18653/v1/2021.emnlp-main.446
- [11] Xiaojie Gu, Guangxu Chen, Jungang Li, Jia-Chen Gu, Xuming Hu, and Kai Zhang. 2025. UltraEdit: Training-, Subject-, and Memory-Free Lifelong Editing in Large Language Models. *arXiv preprint arXiv:2505.14679* (2025).
- [12] Tom Hartvigsen, Swami Sankaranarayanan, Hamid Palangi, Yoon Kim, and Marzyeh Ghassemi. 2024. Aging with grace: Lifelong model editing with discrete key-value adaptors. *Advances in Neural Information Processing Systems* 36 (2024).
- [13] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).
- [14] Hang Li. 2017. Deep learning for natural language processing: advantages and challenges. *National Science Review* 5, 1 (09 2017), 24–26. [arXiv:https://academic.oup.com/nsr/article-pdf/5/1/24/31567231/nwx110.pdf](https://academic.oup.com/nsr/article-pdf/5/1/24/31567231/nwx110.pdf) doi:10.1093/nsr/nwx110
- [15] Xiaopeng Li, Shanwen Wang, Shasha Li, Shezheng Song, Bin Ji, Jun Ma, and Jie Yu. 2025. Rethinking the Residual Distribution of Locate-then-Editing Methods in Model Editing. *arXiv preprint arXiv:2502.03748* (2025).
- [16] Zherui Li, Houcheng Jiang, Hao Chen, Baolong Bi, Zhenhong Zhou, Fei Sun, Junfeng Fang, and Xiang Wang. 2025. Reinforced Lifelong Editing for Language Models. *arXiv preprint arXiv:2502.05759* (2025).
- [17] Bang Liu, Xinfeng Li, Jiayi Zhang, Jinlin Wang, Tanjin He, Sirui Hong, Hongzhang Liu, Shaokun Zhang, Kaitao Song, Kunlun Zhu, et al. 2025. Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. *arXiv preprint arXiv:2504.01990* (2025).
- [18] Vittorio Mazzia, Alessandro Pedrani, Andrea Caciolai, Kay Rottmann, and Davide Bernardi. 2023. A survey on knowledge editing of neural networks. *arXiv preprint arXiv:2310.19704* (2023).
- [19] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and Editing Factual Associations in GPT. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 17359–17372.
- [20] Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2022. Mass-Editing Memory in a Transformer. <http://arxiv.org/abs/2210.07229>
- [21] Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D Manning. 2022. Fast Model Editing at Scale. In *International Conference on Learning Representations*. <https://openreview.net/pdf?id=0DcZxwFOPt>
- [22] Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D Manning, and Chelsea Finn. 2022. Memory-based model editing at scale. In *International Conference on Machine Learning*. PMLR, 15817–15831.
- [23] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, et al. 2020. KILT: a benchmark for knowledge intensive language tasks. *arXiv preprint arXiv:2009.02252* (2020).
- [24] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*. 1631–1642.
- [25] Chenmian Tan, Ge Zhang, and Jie Fu. 2024. Massive Editing for Large Language Models via Meta Learning. *arXiv:2311.04661 [cs.CL]* <https://arxiv.org/abs/2311.04661>
- [26] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, et al. 2024. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118* (2024).
- [27] Alex Wang. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. *arXiv preprint arXiv:1804.07461* (2018).
- [28] Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. <https://github.com/kingoflolz/mesh-transformer-jax>.
- [29] Peng Wang, Zexi Li, Ningyu Zhang, Ziwen Xu, Yunzhi Yao, Yong Jiang, Pengjun Xie, Fei Huang, and Huajun Chen. 2024. WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models. *arXiv:2405.14768 [cs.CL]* <https://arxiv.org/abs/2405.14768>
- [30] Peng Wang, Ningyu Zhang, Bozhong Tian, Zekun Xi, Yunzhi Yao, Ziwen Xu, Mengru Wang, Shengyu Mao, Xiaohan Wang, Siyuan Cheng, Kangwei Liu, Yuansheng Ni, Guozhou Zheng, and Huajun Chen. 2024. EasyEdit: An Easy-to-use Knowledge Editing Framework for Large Language Models. *arXiv:2308.07269 [cs.CL]* <https://arxiv.org/abs/2308.07269>
- [31] Xiaohan Wang, Shengyu Mao, Ningyu Zhang, Shumin Deng, Yunzhi Yao, Yue Shen, Lei Liang, Jinjie Gu, and Huajun Chen. 2024. Editing conceptual knowledge for large language models. *arXiv preprint arXiv:2403.06259* (2024).
- [32] A Warstadt. 2019. Neural Network Acceptability Judgments. *arXiv preprint arXiv:1805.12471* (2019).
- [33] Adina Williams, Nikita Nangia, and Samuel R Bowman. 2017. A broad-coverage challenge corpus for sentence understanding through inference. *arXiv preprint arXiv:1704.05426* (2017).
- [34] Yunzhi Yao, Peng Wang, Bozhong Tian, Siyuan Cheng, Zhoubo Li, Shumin Deng, Huajun Chen, and Ningyu Zhang. 2023. Editing Large Language Models: Problems, Methods, and Opportunities. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 10222–10240. doi:10.18653/v1/2023.emnlp-main.632
- [35] Ningyu Zhang, Yunzhi Yao, Bozhong Tian, Peng Wang, Shumin Deng, Mengru Wang, Zekun Xi, Shengyu Mao, Jintian Zhang, Yuansheng Ni, Siyuan Cheng, Ziwen Xu, Xin Xu, Jia-Chen Gu, Yong Jiang, Pengjun Xie, Fei Huang, Lei Liang, Zhiqiang Zhang, Xiaowei Zhu, Jun Zhou, and Huajun Chen. 2024. A Comprehensive Study of Knowledge Editing for Large Language Models. *arXiv:2401.01286 [cs.CL]* <https://arxiv.org/abs/2401.01286>
- [36] Taolin Zhang, Qizhou Chen, Dongyang Li, Chengyu Wang, Xiaofeng He, Longtao Huang, Hui Xue, and Jun Huang. 2024. DAFNet: Dynamic Auxiliary Fusion for Sequential Model Editing in Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2024*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 1588–1602. doi:10.18653/v1/2024.findings-acl.92
- [37] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).
- [38] Ce Zheng, Lei Li, Qingxiu Dong, Yuxuan Fan, Zhiyong Wu, Jingjing Xu, and Baobao Chang. 2023. Can We Edit Factual Knowledge by In-Context Learning?. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 4862–4876. doi:10.18653/v1/2023.emnlp-main.296
- [39] Chen Zhu, Ankit Singh Rawat, Manzil Zaheer, Srinadh Bhojanapalli, Daliang Li, Felix Yu, and Sanjiv Kumar. 2020. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363* (2020).

A Editing Performance vs. Number of Sequences/Batches

We conduct experiments with RLEdit, RLEdit², MALMEN, MALMEN², and EMSEdit under both sequential and batch editing settings. The results are presented in Figures 9 and 10, respectively.

In sequential editing, RLEdit² and EMSEdit achieve comparable editing performance across different datasets, models, and editing

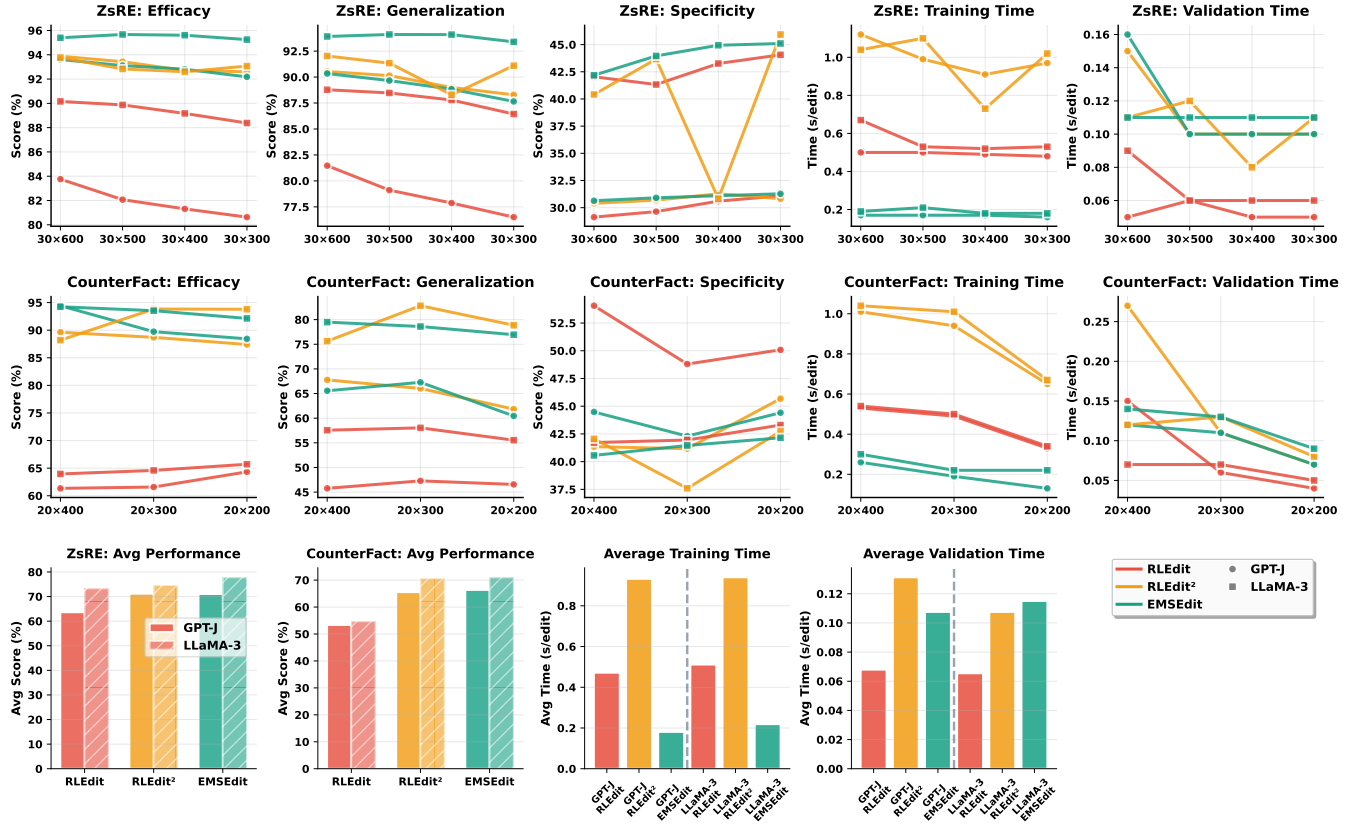


Figure 9: Editing performance of EMSEdit and baselines with respect to the number of editing sequences.

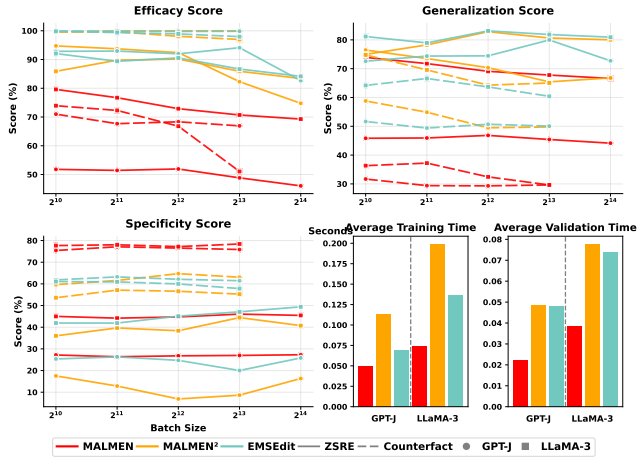


Figure 10: Editing performance of EMSEdit and baselines with respect to the number of editing batches.

setups, both significantly outperforming RLEdit, except in terms of Specificity on the CounterFact dataset. Notably, EMSEdit demonstrates slightly better average performance than RLEdit². In terms of efficiency, RLEdit achieves the highest validation efficiency, which is intuitive since it performs editing using only single-step updates,

whereas EMSEdit and RLEdit² rely on two-step updates. Regarding training efficiency, EMSEdit performs the best, followed by RLEdit, while RLEdit² shows the lowest efficiency. The batch editing results are consistent with those of sequential editing. EMSEdit and MALMEN² achieve comparable overall performance; however, both MALMEN² and EMSEdit show a noticeable drop in Specificity on the CounterFact dataset compared with MALMEN.

Across both sequential and batch settings, the results suggest that multi-step backpropagation (MSBP) improves Efficacy and Generalization but degrades Specificity. This degradation likely arises from overfitting to the injected knowledge. We leave mitigating this issue to future work.

B Analysis of the Loss Variation

In this part, we analyze the trend of training loss across different BP steps in EMSEdit's MSBP process. Since the differences in loss across BP steps are more pronounced in the batch editing experiments, we focus our analysis on this setting. Specifically, we record the loss at each BP step in the batch editing experiment. We show the loss trend in Figure 11. It can be observed that for both datasets and models, the loss at BP Step 2 is consistently lower than at BP Step 1. This indicates that the introduction of MSBP in MLME has a similar effect to multi-epoch BP in standard deep learning training, where the loss gradually decreases with more iterations. We also

Comparison of Gradients and Pseudogradients Across Step 1 and Step 2 (t-SNE)

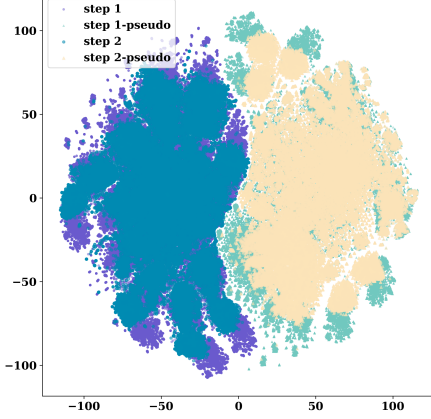


Figure 13: t-SNE visualization of gradients and pseudo-gradients after dimensionality reduction.

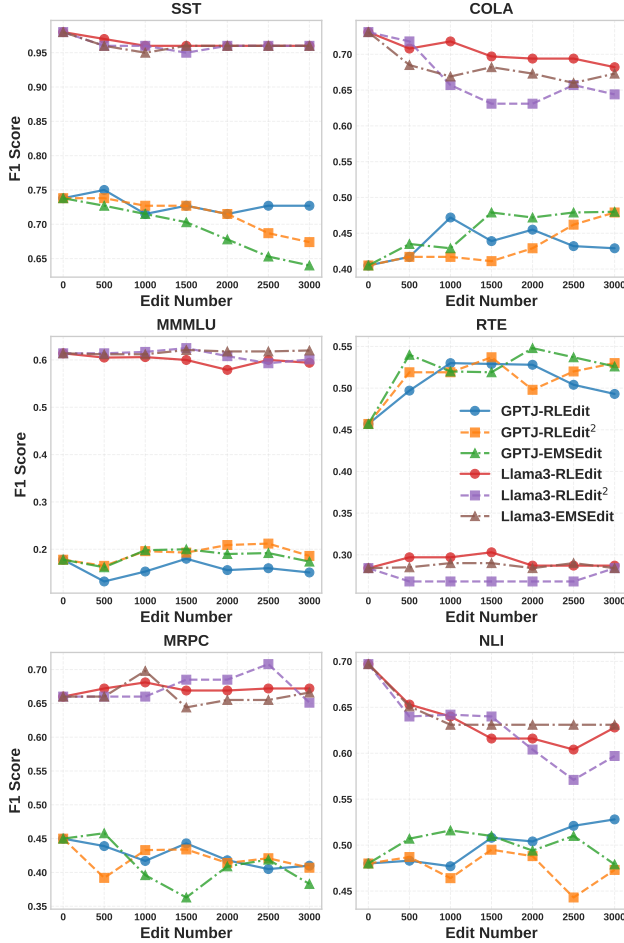


Figure 14: F1 scores of the post-edited LLMs on six tasks.

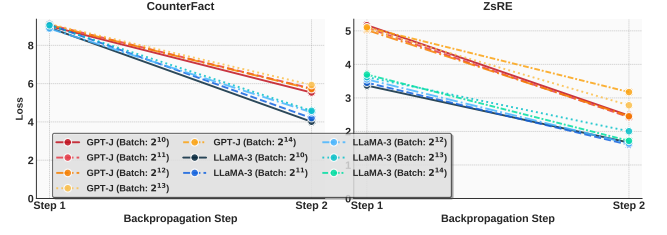


Figure 11: The variation of loss across different BP steps for different batch edit sizes.

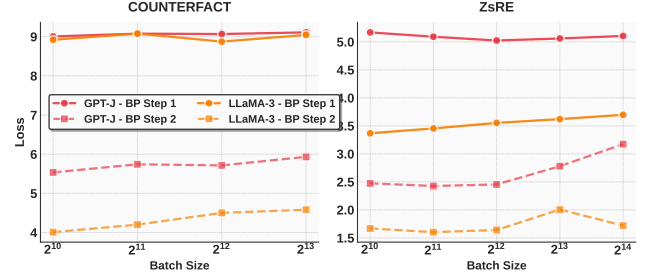


Figure 12: The loss at different BP steps varies with the number of batch edits.

present the variation of the loss norm across different BP steps with respect to the editing batch size. As shown in Figure 12, it can also be observed that for both datasets and models, the loss at BP Step 2 is consistently lower than at BP Step 1. Moreover, the overall increase in loss with larger batch sizes aligns with intuition. This indicates that the introduction of MSBP in MLME has a similar effect to multi-step BP in standard deep learning training, where the loss gradually decreases with more iterations.

Additionally, we visualize the gradients and pseudo-gradients at different BP steps during the validation of 2^{13} batch edits, as shown in Figure 13. It can be observed that there are clear differences between the gradients and pseudo-gradients at both Step 0 and Step 1, which are attributed to the transformation effect of the trained hypernetwork. On the other hand, the distributions of gradients and pseudo-gradients at Step 1 are more dispersed compared to those at Step 2. This indicates that MSBP reduce the spread of the gradient distribution, which is consistent with our previous analysis of the loss across different BP steps.

C General Capability Test

In Section 6.2 and Section 6.2.3, we find that although EMSEdit and MSBP exhibit strong performance in Efficacy and Generalization, they show suboptimal performance in Specificity. To further investigate the impact of EMSEdit and MSBP on the original capabilities of LLMs, we examine the performance of the edited models on general tasks. Following the work of [8, 16], we evaluate the post-edit performance of LLMs using six natural language tasks from the General Language Understanding Evaluation (GLUE) benchmark [27]. The six tasks are: SST (The Stanford Sentiment Treebank) [24], MRPC (Microsoft Research Paraphrase Corpus) [6], MMLU (Massive Multi-task Language Understanding) [13], RTE (Recognizing

Textual Entailment) [3], CoLA (Corpus of Linguistic Acceptability) [32], and NLI (Natural Language Inference) [33].

We perform 30 sequential edits with a batch size of 100 on both GPT-J and LLaMA-3. The performance of the LLMs before and after editing is evaluated every 5 edits. The results are shown in Figure 14. It can be observed that the overall performance of the edited LLMs is comparable across the three methods: EMSEdit, RLEdit, and RLEdit². In some cases, EMSEdit outperforms the other two methods: on the CoLA and RTE tasks with GPT-J, and on the RTE, MMLU, and NLI tasks with LLaMA-3. However, on the SST task with GPT-J, EMSEdit underperforms compared to the other two methods. Despite this, these results suggest that **the introduction of MSBP do not significantly impact the ability of editing methods to preserve the original capabilities of LLMs.**

D Experimental Details

Table 3: Hyperparameter Settings of Model Editing Methods

Hyperparameter	SMEdit	MALMEN	RLEdit	MALMEN ²	RLEdit ²
Rank of linear transformation in hyper-network	1024	1024	1024	1024	1024
Number of blocks in hyper-network	4	4	4	4	4
Initial learning rate	1e-6	1e-6	1e-6	1e-6	1e-6
Meta-learning rate	1e-5	1e-5	1e-5	1e-5	1e-5
Locality coefficient	-	0.6	0.6	0.6	0.6
Maximum meta gradient norm	1	1	1	1	1
Regularization coefficient	0.5	-	1e-4	-	1e-4
Discount factor	-	-	1	-	1
Backtracking depth	-	-	10	-	10
Memory backtracking decay factor	-	-	0.95	-	0.95
BP Steps	2	1	1	2	2

Experiments are conducted on A800 (80G) GPUs and a server equipped with two Intel(R) Xeon(R) Platinum 8358 CPUs @ 2.60GHz.

D.1 Hyperparameter Setting

To ensure a fair comparison, the hyperparameter settings of **EMSEdit**, **MALMEN**, **UltraEdit**, **MEND**, and **RLEdit** are kept largely consistent. For the variant methods **MALMEN²**, **RLEdit²**, and **UltraEdit²**, all hyperparameters remain the same as their original versions, except for the BP steps. For FT, we follow the settings in EasyEdit [30]. We present the hyperparameters of each method in Table 3.

In our study, for **sequential editing**, all methods edit the following modules:

- **GPT-J**: `mlp.fc_in[10,12,14,16,18,20,22,24,26]`
- **LLaMA-3**: `mlp.gate_proj[11-15]` and `mlp.up_proj[18-24]`
- **Gemma-2**: `mlp.gate_proj[32-40]` and `mlp.up_proj[32-40]`

For **batch editing**, all methods edit:

- **GPT-J**: `mlp.fc_out[22-27]`
- **LLaMA-3**: `mlp.gate_proj[11-15]` and `mlp.up_proj[18-24]`
- **Gemma-2**: `mlp.down_proj[32-40]`

D.2 Details of Dataset

ZsRE (Zero-shot Relation Extraction) [39] ZsRE is originally a relation extraction dataset. Zhu et al. [39] processes the training set from the KILT [23] version of ZsRE by evenly distributing the questions corresponding to each fact into the training and test sets, and filtering out samples that exceed the token limit. Each sample in this dataset consists of a question, a paraphrased version of the question, and a semantically inconsistent local question. The question and its paraphrase are used to evaluate the efficacy and generalization of the editing method, respectively, while the local question is used to assess specificity. The metrics are as follows:

- **Efficacy**: It evaluates the ability of edited model θ_{W_1} to correctly predict the exact editing instance (x, y) :

$$\mathbb{E}[y = \arg \max_{\mathbf{y}} \mathbb{P}_{\theta_{W_1}}(\mathbf{y}|x)] \quad (8)$$

- **Generalization**: It evaluates the ability of edited model θ_{W_1} to correctly predict the semantically consistent instance (x^e, y^e) of the exact editing instance:

$$\mathbb{E}[y^e = \arg \max_{\mathbf{y}} \mathbb{P}_{\theta_{W_1}}(\mathbf{y}|x^e)] \quad (9)$$

- **Specificity**: It evaluates the ability of edited model θ_{W_1} to correctly predict the unrelated instance (x^u, y^u) :

$$\mathbb{E}[y^u = \arg \max_{\mathbf{y}} \mathbb{P}_{\theta_{W_1}}(\mathbf{y}|x^u)] \quad (10)$$

CounterFact [19] The CounterFact dataset originates from the PARAREL [7] dataset, with additional paraphrase prompts and neighborhood prompts added to evaluate generalization and specificity, respectively. Following previous work [16], we use efficacy, generalization, and specificity to evaluate the performance of editing methods on this dataset.

- **Efficacy**: It evaluates whether the edited model θ_{W_1} assigns a higher probability to the edited instance (x, y) compared to the old knowledge $(x, \mathbf{y})$:

$$\mathbb{E}[\mathbb{P}_{\theta_{W_1}}(y|x) > \mathbb{P}_{\theta_{W_1}}(\mathbf{y}|x)] \quad (11)$$

- **Generalization**: It evaluates whether the edited model θ_{W_1} assigns a higher probability to the semantically consistent instance (x^e, y^e) of the edited instance compared to the old knowledge $(x, \mathbf{y})$:

$$\mathbb{E}[\mathbb{P}_{\theta_{W_1}}(y^e|x^e) > \mathbb{P}_{\theta_{W_1}}(\mathbf{y}|x^e)] \quad (12)$$

- **Specificity**: It evaluates whether the edited model θ_{W_1} assigns a higher probability to the unrelated instance (x^u, y^u) instead of the edited instance:

$$\mathbb{E}[\mathbb{P}_{\theta_{W_1}}(y^u|x^u) > \mathbb{P}_{\theta_{W_1}}(y|x^u)] \quad (13)$$

E Impact Statements

Model editing is an emerging field aimed at efficiently correcting errors, outdated information, and harmful content in pretrained language models. However, as a powerful technology, it can also be misused to inject malicious information. We strongly oppose such misuse and encourage the research community to develop defenses against it.